Part 5 Brief · CI eval gate and live evals

mcp-ops-server · 19 Oct 2026 · PRs #6-#11 · Status: done; gpt-6-luna 30/31 for $0.046; Sol run optional

WHAT WAS BUILT

- **An eval gate on every push and PR:** a dedicated CI job runs all 31 golden tasks against the real server and all 12 negative controls. Free (no model, no secrets); any failure fails the build. The report lands in the job summary and as an artifact.
- **A live OpenAI agent** on the same tasks, graded by the same grader. Paid, so it runs only when you trigger it: Actions > Live evals > Run workflow, with the model ID as an input.
- **Live metrics:** pass rate, tokens, cost per task (from prices you pass), model and task latency p50/p99, turn-limit hits, and each final answer (shown, never graded).

HOW IT WORKS

Piece	Role
agents_openai.py	Responses API tool-calling loop. The model sees exactly the tools the server advertises, and every call goes through the harness's recorded call(), so it cannot grade itself.
ci.yml: evals	Replay agent + negative controls on every push and PR. Mark it a required check to block merges.
live-evals.yml	workflow_dispatch only. The key lives in a live-evals environment that only main can use (enforced by GitHub, not by this file). Inputs via env (no script injection), 30-min timeout.

WHY IT WAS DONE THIS WAY

- **Free gate, paid measurement.** Replay is deterministic, so it can block merges. A model's score varies run to run, so live runs report a number and do not fail on it unless you ask (`--min-pass-rate`). An API error does fail the run: it measured nothing.
- **Manual trigger only, main only.** No schedule and no PR trigger, so cost stays under your control, and code that has not been merged never runs with the key.
- **Optional dependency.** openai is a separate install extra imported lazily; the server and the CI gate never install it. Tests use a fake client and the real SDK against a local endpoint: no key.

RESULTS (GPT-6-LUNA, DEFAULT EFFORT, ALL 31 TASKS, ONE RUN EACH)

Run	Passed	Cost*	Task p50 / p99	Failures
Before fixes	29/31	$0.038	5.5 s / 35.8 s	k01, f01: both needed info no tool gave
After fixes	30/31	$0.046	5.1 s / 25.0 s	k01: right slot, but 16 calls (limit 6)

*Upper bound: all input priced at $0.10/1M, though cached input bills at $0.01. All 12 negative controls caught in both runs; every refusal, auth and referral-safety task passed.

WHAT WENT WRONG OR SURPRISED ME (LOGGED IN FAILURES.MD)

- **Codex review: 4x P1, 2x P2, all fixed with tests.** Among them: the real client broke from the 2nd task on (my fake client had no connection pool), and a run where every API call failed was green.
- **The live model found four defects of mine.** Chat Completions rejects tools plus reasoning (moved to the Responses API); the report hid the errors (now shown in full); 'Range at most 31 days' was ambiguous (now exact); two tasks needed information only I had, from reading the seed. I fixed the server or the task, never the grader, and wrote the rule down.

VERIFY IT YOURSELF

```
python -m evals.run       # 31/31 tasks passed; 12/12 negative controls caught
ruff check . && python -m pytest -q   # 174 passed (14 for the live agent loop)
Actions > Live evals > a run > Summary # report, incl. each failed call and its error
```

INTERVIEW TALKING POINT

"Every PR runs a free trajectory eval that blocks merges; real-model runs are a separate, paid measurement graded by the same code. The first live runs mostly found bugs in my own eval, not in the model. The rule I took from it: a task may not need information no tool provides."

NEXT

Your actions: mark **evals**, lint and test as required checks on main; optionally run all 31 on GPT-6.1 Sol for a comparison row. **Part 6 (Ship)** waits for your 'go': deploy, README with these numbers, demo.